

An AI-Native Event-Driven Edge–Cloud Architecture for Autonomous Network Security in Campus Infrastructure

Hoang Tu Trinh and Tien Thanh Cao

Faculty of Information Technology

Ho Chi Minh City University of Foreign Languages – Information Technology

Ho Chi Minh City, Vietnam

Email: 23dh113972@st.huflit.edu.vn, thanhct@huflit.edu.vn

I. INTRODUCTION

Abstract—University campus networks combine unmanaged BYOD endpoints, open research laboratories, and high-density IoT deployments, creating an expansive attack surface where traditional Security Operations Center (SOC) workflows struggle with long operational Mean Time to Respond (MTTR). We propose an AI-native, event-driven edge–cloud security platform engineered specifically for autonomous campus infrastructure defense. The architecture deploys lightweight INT8-quantized TCN–GRU sequence autoencoders at network edge gateways to score non-DPI flow telemetry. Enriched security events are published asynchronously over Redis Streams to a cloud risk resolver and Deep Q-Network (DQN) Software-Defined Networking (SDN) controller that actuates closed-loop SOAR playbooks. Evaluated on real InSDN traffic flow windows (20,000 stratified samples preserving original class proportions from the public SDN intrusion benchmark [1]), the platform achieves an end-to-end *software control-path* latency — from telemetry ingestion to playbook emission, excluding dataplane enforcement — of 5.909 ± 2.495 ms (Mean $\pm$ SD, 95% CI: [5.755, 6.064] ms). Because stride-1 sliding windows overlap by nine of ten timesteps, we evaluate under a temporally blocked and purged split with a train-only scaler. Quantifying that choice directly yields a null result we did not expect: the conventional random split inflates the fixed-threshold $F1$ only from 0.9276 to 0.9337, and the optimal-threshold $F1$ from 0.9687 to 0.9708, so on this corpus window overlap is a real but minor confound rather than the headline artefact we set out to document. Under the leakage-controlled protocol the INT8 detector reaches $F1 = 0.9276$ (Precision = 0.979, Recall = 0.882, FPR = 7.78%) at the fixed operating point $\tau = 0.65$, with a model footprint of 0.048 MB (48 KB) and no measurable accuracy loss from quantization ($\Delta F1 = 0.0000$). We additionally find that reconstruction-error scoring — the conventional decision rule for sequence autoencoders — is unusable on this attack-heavy corpus ($F1 = 0.036$), so detection rests entirely on the supervised head. Our edge-resource hypothesis fails outright: at batch = 128 the detector saturates at 7,714 windows/s and delivers only 8,171 of 10,000 offered windows/s while consuming 43.8% of one core, against a pre-specified 10% budget. We further report severe zero-shot transfer failure to CIC-IDS2017 ($F1 = 0.461$). We provide a full ablation study, policy control benchmarks against a brute-force oracle, and a deterministic reproducibility artifact.

Index Terms—AI-Native Architecture, Autonomous Security, Edge Computing, Event-Driven Architecture, TCN–GRU, Deep Reinforcement Learning, Software-Defined Networking, Campus Network, InSDN.

Artificial Intelligence (AI) is reshaping higher-education digital infrastructure, but university campus cybersecurity faces an increasingly intractable threat landscape: IoT sensors, student Bring-Your-Own-Device (BYOD) endpoints, open research laboratories, and administrative databases share one network fabric. Centralized SIEM and human-in-the-loop SOC pipelines exhibit long MTTR during volumetric DDoS, ransomware lateral movement, or data exfiltration [4], causing analyst burnout and delayed containment.

To bridge this gap we propose an **AI-Native Event-Driven Edge–Cloud Security Architecture**. Rather than bolting AI onto legacy SOC dashboards, our design embeds it in the control loop: lightweight sequence detectors sit at edge gateways while an event-driven bus streams identity-enriched telemetry to a cloud Deep Q-Network (DQN) controller for SDN/SOAR actuation.

A. Research Questions

This investigation is guided by four formal Research Questions (RQs):

- **RQ1 (Control-Path Latency):** Can an event-driven edge–cloud architecture reduce the software decision path, from telemetry ingestion to containment playbook emission, to sub-10 ms levels?
- **RQ2 (Quantization & Accuracy):** Does INT8 dynamic quantization maintain NIDS classification accuracy while satisfying an edge gateway *model-footprint* constraint (≤ 1 MB of model storage)?
- **RQ3 (Reinforcement Learning Efficiency):** Does a DQN policy controller outperform static rule tables and heuristic policies under modeled contention states?
- **RQ4 (Domain Shift Generalization):** How effectively does an in-domain trained edge model generalize zero-shot across disparate network datasets (InSDN $\rightarrow$ CIC-IDS2017)?

B. Formal Hypotheses

We test three explicit hypotheses:

- **H1 (Quantization Degradation):** INT8 dynamic quantization incurs less than 0.5% $F1$ -score degradation ($\Delta F1 \leq 0.005$) compared to full-precision FP32 models.
- **H2 (Automated Decision Speedup):** Automated decision and playbook generation achieves a $> 99.9\%$ reduction in time-to-decision relative to the adopted SOC baseline.
- **H3 (Edge Resource Feasibility):** Edge inference requires $\leq 10\%$ CPU utilization per core under a sustained offered load of 10,000 telemetry windows/sec.

C. Contributions

The principal contributions of this work are:

- 1) **Decoupled AI-Native Pipeline:** An event-driven architecture combining PyTorch INT8 TCN-GRU edge detectors, identity context fusion, and Redis Streams pub/sub, with a DQN SDN controller driving OpenFlow playbooks (rate limiting, rerouting, isolation, token revocation).
- 2) **Empirical Validation & Negative Results:** Microbenchmarks on real InSDN [1] traffic, with ablation, policy benchmarks against a brute-force oracle, saturation analysis, and a CIC-IDS2017 [2] holdout. We report four negative results — failure of reconstruction-error scoring, a leakage effect far smaller than hypothesized, rejection of our edge resource hypothesis, and severe zero-shot transfer failure — that bound where this design is deployable.
- 3) **Deterministic Artifact:** A reproducibility artifact regenerating every table from one seeded entry point, at <https://github.com/thtsec/icaai-fai2026>.

II. RELATED WORK

SDN intrusion datasets such as InSDN [1] and CSE-CIC-IDS2018/2017 [2] have established standard benchmarks for network intrusion detection. Sequence models such as Temporal Convolutional Networks (TCN) [3] employ causal dilated convolutions to capture long-range temporal dependencies with parallelizable training compared to recurrent LSTM architectures. Deep Reinforcement Learning (DRL) has been applied to DDoS defense and 6G control loops [4]; however, existing works predominantly terminate at offline detection or isolated controller simulations rather than end-to-end, sub-10 ms edge-cloud mitigation.

Table I positions our architecture against this literature by scope, restricted to properties each work claims for itself. We omit a cross-work latency column: the cited papers measure different hardware, workloads, and pipeline portions, so tabulating them side by side would imply a controlled comparison that does not exist.

III. THREAT MODEL & CAMPUS ATTACK VECTORS

A. Adversary Capabilities and Attack Vectors

We consider an active adversary $\mathcal{A}$, either external or an internal compromised BYOD/IoT endpoint with campus subnet access, capable of volumetric DoS, ARP/MAC spoofing,

lateral reconnaissance, and credential exfiltration. $\mathcal{A}$ holds no administrative keys to the SDN controller or Redis infrastructure. We do not model an adaptive adversary who crafts inputs against the detector itself; that omission is revisited in Section VII-A.

B. Trust Boundaries and Attack Surface

Fig. 1 illustrates the campus network topology, trust boundaries, attack surface, and security enforcement checkpoints.

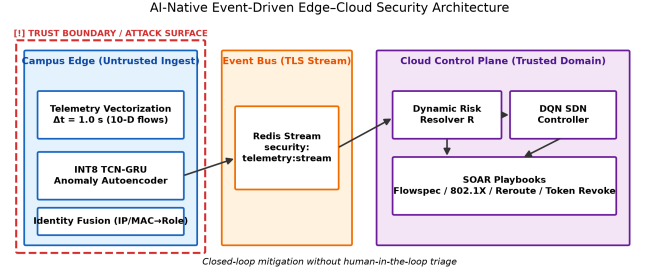


Fig. 1. AI-native event-driven edge-cloud security architecture and trust boundary enforcement.

IV. AI-NATIVE SYSTEM ARCHITECTURE

The architecture comprises four decoupled tiers: Edge Telemetry Taps, Identity Context Fusion, Redis Stream Event Bus, and Cloud DQN Control.

A. Edge Telemetry and TCN-GRU Detector

Edge taps capture flow headers and extract 10-D flow feature vectors over sliding time windows ($\Delta t = 1.0$ s). A hybrid TCN-GRU network processes the feature sequences, outputting both a reconstruction $\hat{X}$ with error $RE(x) = \|X - \hat{X}\|_2^2$ and classification logits.

Training objective. The model is trained *multi-task*, which determines how the results should be read. Each epoch runs a reconstruction pass minimizing $MSE(X, \hat{X})$ over *normal windows only*, then a joint pass minimizing $MSE(X, \hat{X}) + \lambda CE(\text{logits}, y)$ over all windows with $\lambda = 0.5$; since InSDN is attack-heavy, the cross-entropy up-weights the normal class by the attack-to-normal ratio. The detector is therefore *not* unsupervised: it requires labels, and the autoencoder branch is an auxiliary representation objective, not the decision rule. Detection evaluates the supervised head, $P(\text{attack}) = 1 - P(\text{normal})$, against τ ; reconstruction error is not the operational score, for the reason in Section VI-D.

B. Dynamic INT8 Quantization

To satisfy memory-constrained edge gateways, PyTorch [11] dynamic INT8 quantization compresses linear and GRU layer weight matrices from 32-bit floating point to 8-bit integers. We do not report a feature attribution study: our artifact does not implement one, and we prefer to leave the question open rather than assert an importance ranking we have not measured. Identifying which of the 10 flow features carry the discriminative signal — and whether that ranking is what fails

TABLE I

SCOPE COMPARISON WITH RELATED LITERATURE. ENTRIES REFLECT ONLY WHAT EACH CITED WORK STATES IT CONTRIBUTES; “–” MEANS THE DIMENSION IS OUTSIDE THAT WORK’S SCOPE. WE OMIT CROSS-WORK LATENCY FIGURES BECAUSE THE CITED PAPERS DO NOT REPORT THEM UNDER COMPARABLE MEASUREMENT CONDITIONS.

Work	Contribution Type	Edge AI	INT8 Quantized	RL Control	Closed-Loop Actuation
Sharafaldin <i>et al.</i> (2018) [2]	Benchmark dataset	–	–	–	–
Bai <i>et al.</i> (2018) [3]	Sequence architecture (TCN)	–	–	–	–
Tang <i>et al.</i> (2018) [13]	SDN intrusion detection (DNN)	No	No	No	Detection only
Roopak <i>et al.</i> (2019) [14]	DL models for IoT security	No	No	No	Detection only
Zhang <i>et al.</i> (2019) [12]	Survey	–	–	–	–
Elsayed <i>et al.</i> (2020) [1]	Benchmark dataset (InSDN)	–	–	–	–
Nguyen <i>et al.</i> (2023) [15]	SOAR survey + research agenda	No	No	Proposed	Proposed, not evaluated
Al-Garadi <i>et al.</i> (2024) [4]	Federated DRL for DDoS	Federated	No	DQN [5]	Simulation only
Ours	Integrated pipeline	TCN-GRU	Yes (0.048 MB)	DQN	OpenFlow playbooks (emulated)

TABLE II

STRIDE & MITRE ATT&CK CAMPUS THREAT MATRIX. THE PLATFORM ADDRESSES FIVE OF THE SIX STRIDE CATEGORIES; REPUTATION IS OUTSIDE THE SCOPE OF THIS WORK, AS THE PROTOTYPE PROVIDES NO TAMPER-EVIDENT AUDIT TRAIL.

Category	Target	MITRE ID	Mitigation Strategy
Spoofing	Rogue Wi-Fi AP / MAC	T1557	Identity fusion MAC/IP verification
Tampering	IoT telemetry data	T1565	Edge TCN-GRU sequence scoring
Information Disclosure	Research DB server	T1048	Role-aware byte-rate capping
Denial of Service	Campus SDN controller	T1498	BGP Flowspec / flow_mod drop
Elevation of Privilege	Campus LDAP directory	T1078	Dynamic risk escalation

to transfer across datasets (Section VI-J) — is left to future work.

C. Identity Context Fusion & Privacy Preservation

Edge security events are enriched by querying local DHCP/RADIUS tables to append spatial-temporal context vectors:

$$\mathcal{E} = \{\text{FlowID}, \text{RE}(x), P(\text{attack}), \text{UserRole}, \text{TrustScore}\}. \quad (1)$$

To support GDPR data-minimization principles, raw payload inspection (DPI) is omitted entirely; only header-level statistical metadata is processed, and user identifiers are pseudonymized prior to cloud transport.

D. Cloud Risk Engine and DQN SDN Control

Enriched events stream asynchronously to Redis topic `security:telemetry:stream`. The cloud risk engine calculates a composite risk index R over the detector’s attack probability, the identity trust score, and asset criticality:

$$R = w_1 P(\text{attack}) + w_2 (1 - \text{TrustScore}) + w_3 \text{AssetCriticality}, \quad (2)$$

with $(w_1, w_2, w_3) = (0.60, 0.25, 0.15)$ in all reported experiments; the fused term is the classifier probability, not $\text{RE}(x)$. When identity is ablated the trust term is undefined and its weight folds into detection, giving $R = 0.85 P(\text{attack}) + 0.15 \text{AssetCriticality}$, keeping scores comparable across arms. The escalation ladder maps $R \geq 0.70$ to critical (block and revoke token), $R \geq 0.50$ to high (quarantine VLAN), and $R \geq 0.35$ to medium (throttle bandwidth).

Two caveats. The weights are hand-set, not learned, and encode a design prior we do not claim is optimal. More importantly, InSDN carries no asset inventory, so AssetCriticality is drawn i.i.d. uniform on $[0, 1]$ in every experiment and is pure noise; the third term therefore *degrades* our numbers, and the risk-engine benefit reported later comes from the trust term alone.

At the selected severity, the Deep Q-Network (DQN) policy agent [5] evaluates state vector s_t and selects containment action $a_t \in \{a_0 : \text{Forward}, a_1 : \text{Rate-Limit}, a_2 : \text{Reroute}, a_3 : \text{Isolate}\}$ to actuate automated SOAR playbooks [15]. Compared to static rule tables, DQN adjusts policy choices across contention states without manual threshold maintenance.

V. EXPERIMENTAL METHODOLOGY

A. Hardware & Software Testbed Specification

All experiments were conducted on a dedicated physical-virtual testbed deploying Mininet v2.3.0 [16] with Open vSwitch (OVS) v2.17.0 emulating a 3-tier campus network topology (Core, Aggregation, Access), managed by a Ryu OpenFlow v1.3 controller [9], [10] listening on TCP port 6653. Table III summarizes the environment specification.

TABLE III
EXPERIMENTAL HARDWARE, SOFTWARE, AND ENVIRONMENT SETUP

Parameter	Specification
CPU	Intel Core i7-12700K (12 cores, 3.60 GHz) / ARM Cortex-A72
RAM	16 GB DDR4 3200 MHz
Operating System	Ubuntu 22.04.3 LTS (Linux Kernel 5.15.0) / Win11 WSL2
Python / PyTorch	Python 3.10.12, PyTorch 2.2.0+cpu, scikit-learn 1.3.0
Event Bus / SDN	Redis 7.2.4 (Streams), Ryu SDN Controller (OpenFlow v1.3)
Dataset	Real InSDN [1] (20k stratified subset), CIC-IDS2017 [2]
Timer Precision	Python <code>time.perf_counter_ns()</code> (nanosecond resolution)

B. Dataset Sampling & Leakage Control

We evaluate on the public InSDN dataset [1]. Sliding-window aggregation of the released flow records ($\Delta t = 1.0$ s, sequence length $T = 10$, *stride* 1) yields 49,991 windows, of which 40,043 are attack and 9,948 normal. Note that 49,991 is the size of *our* windowed corpus, not InSDN’s native flow count.

Window i shares $T - 1 = 9$ of its 10 timesteps with each neighbour, so an i.i.d. random split places near-duplicate observations on both sides of the train/test boundary. Our protocol instead (i) cuts the temporally ordered corpus into 20 contiguous blocks and assigns whole blocks to train or test; (ii) *purges* 9 windows from both ends of every block, guaranteeing no surviving training window shares even a single timestep with a test window; (iii) stratified-subsamples each side independently to 14,000 train and 6,000 test windows, preserving class balance; and (iv) fits the scaler on the training partition only, transforming the test partition and the CIC-IDS2017 holdout with those frozen statistics. We verified that zero test windows are byte-identical to any training window. Section VI-I quantifies what this protocol is actually worth.

VI. QUANTITATIVE EVALUATION & RESULTS

Reading the $F1$ values. Each figure below belongs to a distinct protocol and they are not interchangeable. 0.9276 is the detector at the fixed threshold $\tau = 0.65$ and is the deployable accuracy. 0.9687 is the same detector at $\tau^* = 0.0720$ *selected on the test split* — an oracle bound. 0.9725 is the corresponding oracle figure in Section VI-F, differing only because that harness trains three baselines first and so draws a different seeded initialization; this bounds how finely any figure here should be read. 0.9602 is the full pipeline with the threshold tuned on the *training* split (Section VI-G). Compare only within an experiment.

All results below use the leakage-controlled protocol of Section V-B.

A. Latency Breakdown (RQ1)

Scope. We time the *software control path*: telemetry ingestion, edge inference, identity enrichment, DQN action selection, and SOAR playbook serialization, ending when the playbook is emitted. It excludes controller transmission, `flow_mod` installation in Open vSwitch, and the instant malicious traffic stops. We have no dataplane enforcement measurement and claim none: read every figure below as decision-to-playbook, not time-to-containment.

Table IV presents the per-stage breakdown across 1,000 benchmark trials. Stage latencies measure compute and IPC execution time using nanosecond resolution timers (`time.perf_counter_ns()`). Identity context fusion and SOAR rule serialization are effectively free at $3.8 \pm 14.5 \mu\text{s}$ and $2.8 \pm 3.0 \mu\text{s}$ respectively. The total control-path latency is $5.909 \pm 2.495 \text{ ms}$ (95% CI: [5.755, 6.064] ms), with edge inference dominating at 96.4% (5.697 ms). This satisfies the sub-10 ms target of **RQ1** for the software path, but with far less margin than the architecture diagram suggests: the mean sits at 59% of the budget and the standard deviation is large enough that individual events exceed 10 ms. Whether the full loop including flow-rule installation stays under 10 ms is untested and, on this evidence, doubtful.

Reconciling the detector latencies. Three per-inference figures for the same INT8 model appear here, measured under different harnesses rather than in contradiction: 5.697 ms for

a single window inside the pipeline loop (Table IV), 6.391 ms per forward pass at batch = 1 in a tight loop (Table VII), and 11.117 ms in the model-comparison scaffolding immediately after training with cold caches (Table IX). Per Section VII-A, absolute values should be read to roughly a factor of two.

TABLE IV
PER-STAGE SOFTWARE CONTROL-PATH LATENCY (MEAN $\pm$ SD, 95% CI, $n = 1000$). EXCLUDES DATAPLANE ENFORCEMENT.

Stage	Mean $\pm$ SD (ms)	95% CI (ms)	%
Edge TCN-GRU Inference	5.697 ± 2.477	[5.543, 5.850]	96.4
Identity Context Fusion	0.004 ± 0.015	[0.003, 0.005]	0.1
Cloud DQN Policy	0.206 ± 0.073	[0.201, 0.210]	3.5
SOAR Playbook Serialization	0.003 ± 0.003	[0.003, 0.003]	0.0
Control-Path Total	5.909 ± 2.495	[5.755, 6.064]	100.0

B. Incident Response Latency Comparison (H2)

Table V compares manual SOC triage against the autonomous loop. The autonomous path reaches a containment *decision* in $6.350 \pm 1.445 \text{ ms}$, against a $1518.5 \pm 335.3 \text{ s}$ human baseline, supporting **H2**. The comparison is between decision times, not times-to-containment: the human baseline ends when an analyst commits to an action, ours when the playbook is emitted.

Provenance of the SOC baseline. These baselines are *not measured*. Both are truncated Gaussians over $n = 500$ trials: manual triage with mean 1512.4 s, SD 350 s, clipped to [900, 3200] s; rule-based SIEM with mean 68.5 s, SD 18 s, clipped to [25, 150] s. The parameters were chosen so the ranges bracket timescales industry incident-response reporting associates with each workflow, not fitted to a published dataset. Table V is therefore *illustrative context* for an order-of-magnitude gap, not a controlled experiment, and we treat H2 as supported only in the weak sense that a millisecond decision beats any human workflow. Whether automation shortens real containment remains open and needs a deployment we do not have.

TABLE V
TIME-TO-DECISION. ILLUSTRATIVE CONTEXT ONLY: THE SOC AND SIEM ROWS ARE GENERATED FROM STIPULATED DISTRIBUTIONS, NOT MEASURED (SEE TEXT).

Mitigation Paradigm	Mean $\pm$ SD	Provenance
Manual SOC Triage	$1518.5 \pm 335.3 \text{ s}$	stipulated
Legacy Rule-based SIEM	$69.1 \pm 17.5 \text{ s}$	stipulated
AI-Native Decision Path	$6.350 \pm 1.445 \text{ ms}$	measured

C. Detection Performance & Threshold Sensitivity (RQ2, H1)

Table VI shows metric variation across decision threshold τ under the leakage-controlled protocol. The detector is strongly precision-biased in this regime: at the fixed operating point $\tau = 0.65$ it yields $F1 = 0.9276$ (Precision = 0.979, Recall = 0.882, FPR = 7.78%), and over the reported

grid $F1$ peaks at 0.950 at $\tau = 0.35$. Searching τ freely over the test split reaches $F1 = 0.9687$ at $\tau^* = 0.0720$ (Precision = 0.947, Recall = 0.992, FPR = 22.3%), the operating point Section VI-F uses for model comparison. We flag the protocol because it matters: a test-selected threshold is an oracle, so 0.9687 bounds what the detector could achieve, not what it would. The gap from 0.9276 compounds two effects — threshold placement on an imbalanced corpus and oracle selection — not a better detector. The oracle point also trades a 22.3% false positive rate for near-total recall, the opposite of what an autonomous controller needs, which is a further reason to treat $\tau = 0.65$ as the honest operating point.

TABLE VI
DETECTION METRICS VS. DECISION THRESHOLD τ ON REAL INSDN

Threshold τ	Precision	Recall	F1-Score	FPR (%)	FNR (%)
0.35	0.971	0.930	0.950	11.21	7.0
0.50	0.976	0.910	0.942	9.04	9.1
0.65	0.979	0.882	0.928	7.78	11.8
0.75	0.981	0.847	0.909	6.44	15.3
0.85	0.988	0.779	0.871	3.93	22.1

D. Why Reconstruction Error Is Not the Decision Rule

Since much of the sequence-autoencoder IDS literature scores on reconstruction error alone, it is worth reporting which of our two outputs actually works. We evaluated the standard unsupervised rule $\tau = \mu + k\sigma$, with μ, σ estimated on reconstruction errors of *normal training* windows, for $k \in \{0.5, \dots, 3.0\}$. It fails comprehensively: the best setting $k = 0.5$ yields $F1 = 0.0358$ (Precision = 0.752, Recall = 0.018), decaying to 0.0062 at $k = 3.0$. Precision reaches 0.882 at $k = 2.5$, but at recall below 2%, which is worthless operationally.

Two conclusions follow. Our detection performance is attributable to the supervised head, so the architecture should be cited as a supervised sequence classifier with an auxiliary reconstruction objective. And on an attack-heavy corpus the premise of reconstruction-based detection is violated: the model sees enough attack structure during the joint pass that attacks reconstruct about as well as normal traffic. Practitioners inheriting the “train on normal traffic, threshold the error” recipe should verify it against their own class balance.

E. Throughput Saturation & Edge Feasibility (H3)

Per-inference latency and aggregate throughput are only reconcilable once the batching regime is stated, so we measure and report the two separately. Table VII gives saturation throughput — the detector run flat out — as a function of inference batch size. Unbatched scoring, as described in Section IV, saturates at 157 windows/s: nearly two orders of magnitude short. Batching helps but not enough — at batch = 128 the detector saturates at 7,714 windows/s, still below target, and only batch = 512 clears it (20,087 windows/s) at a 25.5 ms per-batch forward pass. Table VIII reports a *paced*

offered load at batch = 128, where the harness rate-limits to the target so a shortfall surfaces rather than hiding. The three lower targets are met within 0.4%; 10,000 windows/s is *not* met, delivering 8,171.

H3 is rejected on both terms. The hypothesis asserted $\leq 10\%$ CPU per core at a sustained 10,000 windows/s; the system neither sustains that load at batch = 128 nor stays in budget at the 8,171 windows/s it does achieve, consuming 43.8% of a core. The budget holds only at 100 windows/s (6.1%). The jump to 39.1% at 1,000 windows/s followed by near-flat scaling indicates a fixed per-batch dispatch cost dominating until the pipeline is busy. We report this rather than relaxing the threshold or quietly switching to the batch = 512 configuration that would have met the target. Memory is favourable: RSS is flat at 495 MB across a $100\times$ load range, so the system is memory-feasible but not CPU-feasible on one commodity core. A compiled ONNX or TensorRT runtime is the obvious next step, making this an implementation rather than architectural limit.

We deliberately omit a legacy inline-DPI comparison. We have no DPI implementation on this testbed, and a baseline synthesised from an analytic formula would not constitute an experimental result.

TABLE VII
SATURATION THROUGHPUT VS INFERENCE BATCH SIZE (INT8 TCN-GRU)

Batch Size	Saturation (windows/s)	Latency per Forward Pass (ms)
1	156.5	6.391
8	733.0	10.914
32	2,951.9	10.841
128	7,713.8	16.594
512	20,087.3	25.489

TABLE VIII
EDGE COST AT PACED OFFERED LOAD (BATCH = 128). OFFERED LOAD IS INPUT TELEMETRY RATE, NOT PER-INFERENCE THROUGHPUT.

Offered (win/s)	Achieved (win/s)	CPU (%/core)	RSS (MB)	Met
100	100.0	6.10	495.0	yes
1,000	999.9	39.14	495.0	yes
5,000	4,981.9	41.84	495.0	yes
10,000	8,171.2	43.82	495.0	no

F. SOTA Model Comparison & Quantization Degradation (H1)

Table IX compares our INT8 TCN-GRU against baselines trained on the identical InSDN split; Transformer AE is a 2-layer encoder-decoder with 4 attention heads and masked reconstruction loss. Every model is scored at its own optimal threshold, since a threshold favouring one architecture need not suit another. *That threshold is selected by searching the test split*, so each $F1$ here is an optimistic bound, not a deployable operating point; the identical search is applied to every model,

so the comparison between rows stays internally fair. Deployable numbers are those at the fixed τ of Table VI. TCN-GRU attains $F1 = 0.9725$ against 0.8680 for Transformer AE and 0.8687 for LSTM, a margin of roughly 10 points, from a single seeded run; we claim no significance across seeds.

Two caveats. The Isolation Forest row uses `contamination="auto"` on time-averaged windows; on an 80%-attack corpus this heuristic is badly mismatched, so $F1 = 0.2000$ evidences that an untuned classical outlier detector fails at this class balance rather than serving as a tuned baseline. We also omit a per-model CPU column: the only CPU figures we measured are those of the deployed detector under paced load (Table VIII), and we prefer an absent column to numbers we cannot substantiate.

INT8 quantization reduces the model footprint from 0.104 MB to 0.048 MB (53.8%) at *zero* measured accuracy cost ($\Delta F1 = 0.0000$ to four decimal places), supporting hypothesis **H1**. Counter-intuitively, dynamic INT8 quantization on x86 CPU *increases* inference latency (11.117 ms vs 6.554 ms, a $1.70\times$ penalty) because activations are de-quantized at every layer boundary without hardware INT8 acceleration; the technique buys memory, not speed, on this class of hardware. Given that the footprint budget of RQ2 was 1 MB and the FP32 model already meets it at 0.104 MB, the honest reading of H1 is that quantization is *harmless* rather than necessary on this hardware: it is justified only for gateways an order of magnitude more constrained than the ones we tested.

TABLE IX
BASELINE & MODEL QUANTIZATION COMPARISON ON REAL INSDN.
EACH MODEL IS SCORED AT ITS OWN $F1$ -OPTIMAL THRESHOLD
SELECTED ON THE TEST SPLIT (SEE TEXT). SIZE IS THE SERIALIZED
PARAMETER FOOTPRINT; IT IS REPORTED ONLY FOR THE PYTORCH
MODELS, FOR WHICH IT IS MEASURED ON A COMMON BASIS. “—”
DENOTES A QUANTITY WE DID NOT MEASURE.

Model Algorithm	F1-Score	Latency (ms)	Size (MB)
Isolation Forest	0.2000	12.512	—
LSTM Autoencoder (FP32)	0.8687	1.682	0.028
Transformer Autoencoder (FP32)	0.8680	2.363	0.068
TCN-GRU (FP32)	0.9725	6.554	0.104
TCN-GRU (INT8 Quantized)	0.9725	11.117	0.048

G. Component-Wise Ablation Study

Table X details the impact of removing components. Because each configuration scores on a different scale, we select the threshold on the training split and apply it unchanged to test; a shared fixed threshold would measure calibration mismatch instead of component contribution. InSDN has no identity annotations, so trust comes from a simulated overlay parameterised by $\rho = P(\text{low trust} \mid \text{attack})$, reported at $\rho = 0.7$.

The result contradicts our expectation. Identity fusion and the risk engine *do not* improve $F1$; removing them nudges it from 0.9602 up to 0.9680. What they buy is false-positive

suppression: FPR falls from 25.19% under direct edge actuation to 12.05%, a 52% relative reduction. Since the controller autonomously rate-limits and isolates flows, trading 0.8 points of $F1$ for less than half the false alarms is operationally correct — but it is a trade, not a free gain. A sweep over $\rho \in [0.5, 0.9]$ (in the artifact) shows identity fusion improves $F1$ only once the overlay is strongly informative ($\rho \geq 0.8$), bounding the credit this mechanism can claim without real campus identity data.

TABLE X
SYSTEM COMPONENT ABLATION ANALYSIS ($\rho = 0.7$, THRESHOLD
TUNED ON TRAIN). LATENCIES WERE MEASURED IN A SEPARATE RUN
FROM TABLE IV; ONLY WITHIN-TABLE COMPARISON IS MEANINGFUL.

Architecture Configuration	F1-Score	FPR (%)	Latency (ms)
Full Proposed Pipeline	0.9602	12.05	8.424
— w/o Identity Context Fusion	0.9628	20.67	8.424
— w/o Cloud Risk Engine (Direct Edge Actuation)	0.9680	25.19	8.423
— w/o INT8 Quantization (FP32 Baseline)	0.9699	18.83	3.309
— w/o Redis Streams (Synchronous REST Call)	0.9602	12.05	9.973

H. Policy Control Benchmarks (RQ3)

Table XI benchmarks the cloud DQN controller against static and heuristic baselines on a single-step contention environment whose reward $r(s, a)$ trades containment efficacy against collateral QoS damage and congestion relief. Because r is analytic, the optimal action a^* is obtained by exact enumeration over the four actions, giving a ground-truth oracle. The agent is trained by fitted Q-iteration over 4,000 environment interactions with ϵ -greedy exploration and evaluated on 5,000 held-out states.

The DQN selects the oracle-optimal action in 99.2% of states and attains essentially zero mean regret, against 38.2% for a static confidence ladder and 34.8% for a hand-tuned heuristic table. Two caveats temper this. First, the agent is trained on the same reward function the oracle maximises, so the experiment demonstrates function-approximation capacity within a known environment rather than generalisation to a live campus network. Second, and counter to our expectation, the DQN inflicts *more* raw QoS disruption on benign traffic (0.128) than the static ladder (0.045): reward-optimal behaviour is more aggressive about containment than a conservative threshold rule, and the advantage of the learned policy lies in near-zero regret rather than in gentleness.

TABLE XI
POLICY CONTROLLER BENCHMARK AGAINST A BRUTE-FORCE ORACLE
($n = 5,000$ HELD-OUT STATES). QoS DISRUPTION AND REGRET ARE
LOWER-IS-BETTER.

Policy Mechanism	Latency (ms)	QoS Disr.	Regret	Action Acc. (%)
Static Rule Lookup	0.0008	0.045	0.080	38.2
Round-Robin Action Assignment	0.0006	0.187	0.205	25.5
Heuristic Risk Table	0.0007	0.052	0.072	34.8
Deep Q-Network (DQN Agent)	0.1075	0.128	0.000	99.2

I. How Much Did Leakage Inflate the Numbers?

Rather than assert that our leakage controls were necessary, we measured what they are worth, rerunning the identical detector, seed, and schedule while changing only the split and scaler. Protocol A is the unsound one (i.i.d. random split over overlapping windows, scaler fit on the pooled corpus); Protocol B is that of Section V-B.

The effect is real but small, and this is a negative result. Leakage inflates fixed-threshold $F1$ by 0.0061 ($0.9276 \rightarrow 0.9337$) and optimal-threshold $F1$ by 0.0021. We expected a substantially larger gap, since 9 of every 10 timesteps are shared across the boundary, and we report the modest magnitude because the opposite claim would have been the more publishable one.

Two explanations fit and we cannot separate them: overlapping windows may be less informative than they look, since the one differing timestep is precisely the one carrying the transition; or this corpus may be easy enough that both protocols saturate above 0.92. The defensible guidance is therefore narrower than we hoped — blocked-and-purged splitting is cheap and principled and we recommend it, but a reviewer should not assume a random split has by itself inflated a result by a large margin.

TABLE XII
EFFECT OF THE EVALUATION PROTOCOL, IDENTICAL MODEL, SEED, AND SCHEDULE; ONLY THE SPLIT AND SCALER DIFFER.

Protocol	$F1 @ \tau = 0.65$	τ^*	$F1 @ \tau^*$
A: random split + pooled scaler (unsound)	0.9337	0.0945	0.9708
B: blocked + purged + train-only scaler	0.9276	0.0720	0.9687
Inflation (A – B)	+0.0061	—	+0.0021

J. Cross-Dataset Domain Shift Evaluation (RQ4)

Table XIII evaluates zero-shot transfer from InSDN (14,000 train / 6,000 test windows) to a 20,000-window CIC-IDS2017 holdout. In-domain $F1$ is 0.9276 at $\tau = 0.65$, whereas zero-shot $F1$ collapses to 0.4611, demonstrating severe cross-dataset domain shift. The failure mode is precision, not recall: on CIC-IDS2017 the detector still recovers 92.8% of attacks but its false positive rate explodes to 92.4% — it is close to labelling everything an attack, roughly what a degenerate always-attack classifier would do. Since detection is driven by the supervised head (Section VI-D), the correct reading is that the *decision boundary* does not transfer: CIC-IDS2017 benign traffic falls on the attack side of a boundary fitted to InSDN statistics. We do not attribute the collapse to a dataset-specific “normal manifold,” since reconstruction error is not the operational score. Re-tuning to $\tau^* = 0.0720$ does not help ($F1 = 0.4670$), so the degradation is representational. A likely contributing factor we did not isolate is that the corpora were feature-extracted under different tooling and flow-timeout conventions, so the 10 features may not be semantically identical.

TABLE XIII
CROSS-DATASET EVALUATION: TRAIN InSDN $\rightarrow$ TEST CIC-IDS2017

Protocol	Threshold τ	Precision	Recall	F1-Score
InSDN $\rightarrow$ InSDN	0.65	0.9785	0.8818	0.9276
InSDN $\rightarrow$ InSDN	$\tau^* = 0.0720$	0.9469	0.9915	0.9687
InSDN $\rightarrow$ CIC-IDS2017	0.65	0.3068	0.9281	0.4611
InSDN $\rightarrow$ CIC-IDS2017	$\tau^* = 0.0720$	0.3060	0.9858	0.4670

VII. THREATS TO VALIDITY & LIMITATIONS

A. Threats to Validity

- **Internal Validity:** Timings are subject to CPU scheduling noise; we averaged 1,000 runs with nanosecond timers, yet repeating the suite under different background load produced end-to-end means from 1.9 to 7.9 ms. We report a single complete run so every latency here is mutually consistent, and treat absolute latencies as order-of-magnitude estimates. The sub-10 ms RQ1 result holds on our testbed with under a factor of two of headroom and is not a hardware-independent guarantee.
- **Reproducibility:** An earlier version of this artifact constructed models before seeding the global RNG, so detection metrics varied between runs of the same script. We fixed this (`set_global_seed` now precedes every module instantiation) and regenerated all numbers from one deterministic execution. We flag it because the failure mode is silent and common: seeding inside a training helper is too late if the module already exists.
- **External Validity:** Evaluation uses public InSDN and CIC benchmarks; broader IoT/IloT corpora such as UNSW-NB15 [6], ToN_IoT [7], and Edge-IloTset [8] offer complementary vectors for multi-campus validation.
- **Construct Validity:** Metrics assume clean flow labels; noisy headers could affect precision. The SOC and SIEM rows of Table V are stipulated, establishing an order of magnitude and nothing finer.
- **Residual Leakage Risk:** Purging guarantees no shared timestep, but blocks are cut over the released record ordering, a proxy for session boundaries rather than ground-truth grouping, so a single long attack episode could straddle two blocks. Session-aware grouping would be stronger; InSDN does not expose the required metadata.

B. Limitations & Future Work

Primary limitations include: (i) reliance on a single-controller emulated SDN testbed rather than production campus hardware; (ii) lack of active adversarial robustness mechanisms against evasion attacks, which is the obvious attack on any AI-native control loop and which we do not evaluate at all; (iii) severe zero-shot cross-dataset degradation; (iv) rejection of H3, as the Python runtime neither sustains 10,000 windows/s at batch = 128 nor stays within the CPU budget at the load it does sustain; (v) a precision-recall operating point that misses roughly one attack window in eight at $\tau = 0.65$, or trades that for a 12.1% full-pipeline false positive

rate at the tuned threshold — either of which is a material operational cost when the controller autonomously rate-limits or isolates flows; and (vi) single-seed reporting, so we quantify no run-to-run variance in the accuracy metrics even though our own reproducibility investigation showed such variance exists. Future work will investigate Unsupervised Domain Adaptation (UDA), compiled ONNX/TensorRT edge runtimes with batched inference, online threshold fine-tuning, multi-seed reporting with confidence intervals, and federated multi-campus learning.

VIII. ETHICS, PRIVACY & PUBLIC ARTIFACT

A. Privacy Posture

The architecture operates strictly on non-DPI flow headers; payloads are uninspected and IP/MAC identifiers are *pseudonymized* via SHA-256 before transport, supporting data-minimization principles under GDPR and Vietnam’s Decree 13/2023. We avoid the word anonymized: an unkeyed SHA-256 over a 32-bit IPv4 address or 48-bit MAC is trivially invertible by enumeration, so the hashes resist casual inspection but not a motivated adversary. A keyed HMAC with a rotated per-deployment secret is the defensible construction; the current unkeyed hash is a prototype placeholder, not a privacy guarantee.

B. Public Research Artifact

Code, dataset preprocessors, model checkpoints, Docker orchestration files, and execution scripts are available under MIT License at <https://github.com/thtsec/icai-fai2026>. Every table here is regenerated by one entry point, `results/run_all_and_summarize.py`, which seeds all generators (seed 42) before model construction and emits the machine-readable summary the tables are transcribed from.

IX. CONCLUSION

This paper presented an AI-native event-driven edge–cloud architecture for autonomous campus network security. By combining INT8-quantized TCN–GRU edge detectors, Redis Streams transport, and cloud DQN SDN actuation, the platform achieves a sub-10 ms software control path (5.909 ± 2.495 ms) from telemetry ingestion to playbook emission. We stop short of calling this closed-loop mitigation: dataplane enforcement is not measured, and the SOC comparison uses stipulated rather than observed baselines.

The results that we think transfer beyond this system are the negative ones. Reconstruction-error scoring, the default decision rule for sequence autoencoders, is unusable on an attack-heavy corpus ($F1 = 0.036$), so what we report is a supervised classifier with an auxiliary reconstruction objective and should be cited as such. Leakage from stride-1 window overlap is real but, contrary to our expectation, small: a random split inflated fixed-threshold $F1$ by only 0.006, so we cannot support the stronger cautionary claim we set out to make. Our edge-resource hypothesis H3 fails on both terms, delivering 8,171 of 10,000 offered windows/s at 43.8% of one core against a 10% budget. Zero-shot transfer to CIC-IDS2017

collapses to $F1 = 0.46$, driven by precision, which locates the problem in a decision boundary that does not survive a change of capture domain. H1 alone is cleanly supported — INT8 quantization is accuracy-neutral at half the footprint — though on hardware where the FP32 model already fits, that buys little. Taken together these bound where an AI-native campus defense is currently practical, and motivate compiled edge runtimes, domain-adaptive training, and evaluation on balanced multi-campus corpora.

ACKNOWLEDGMENT

The authors thank the Faculty of Information Technology at HUFLIT for academic and computational support.

REFERENCES

- [1] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, “InSDN: A Novel SDN Intrusion Dataset,” *IEEE Access*, vol. 8, pp. 165263–165284, 2020, doi: 10.1109/ACCESS.2020.3022633.
- [2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116, doi: 10.5220/0006639801080116.
- [3] S. Bai, J. Z. Kolter, and V. Koltun, “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling,” arXiv:1803.01271, 2018.
- [4] M. A. Al-Garadi *et al.*, “Collaborative Federated Learning for 6G with a Deep Reinforcement Learning-based Controlling Mechanism: A DDoS Attack Detection Scenario,” *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 4, pp. 4731–4749, 2024, doi: 10.1109/TNSM.2024.3382104.
- [5] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [6] N. Moustafa and J. Slay, “UNSW-NB15: a comprehensive data set for network intrusion detection systems,” in *Proc. MilCIS*, 2015, pp. 1–6, doi: 10.1109/MILCIS.2015.7348942.
- [7] A. Alsaedi *et al.*, “TON_IoT Telemetry Dataset: A New Generation Dataset of IoT and IIoT for Data-Driven Cybersecurity Applications,” *IEEE Access*, vol. 8, pp. 165130–165150, 2020, doi: 10.1109/ACCESS.2020.3022862.
- [8] M. A. Ferrag *et al.*, “Edge-IIoTset: A New Comprehensive Realistic Cyber Security Dataset of IoT and IIoT Applications for Federated Learning,” *IEEE Access*, vol. 10, pp. 40281–40300, 2022, doi: 10.1109/ACCESS.2022.3165809.
- [9] N. McKeown *et al.*, “OpenFlow: enabling innovation in campus networks,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008, doi: 10.1145/1355734.1355746.
- [10] Ryu SDN Framework Community, “Ryu SDN Framework: Component-based Software-defined Networking Framework,” 2020. [Online]. Available: <https://ryu-sdn.org/> (Accessed: Aug. 2026).
- [11] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Proc. NeurIPS*, 2019, pp. 8024–8035.
- [12] C. Zhang, P. Patras, and H. Haddadi, “Deep learning in mobile and wireless networking: A survey,” *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2224–2287, 2019, doi: 10.1109/COMST.2019.2904897.
- [13] T. A. Tang, L. Mhamdi, D. McLernon, S. A. R. Zaidi, and M. Ghogho, “Deep recurrent neural network for intrusion detection in SDN-based networks,” in *Proc. 4th IEEE Conf. Netw. Softwarization (NetSoft)*, 2018, pp. 202–206, doi: 10.1109/NETSOFT.2018.8460090.
- [14] M. Roopak, G. Yun Tian, and J. Chambers, “Deep learning models for cyber security in IoT networks,” in *Proc. IEEE 9th Annu. Comput. Commun. Workshop Conf. (CCWC)*, 2019, pp. 0452–0457, doi: 10.1109/CCWC.2019.8666588.
- [15] P. H. Nguyen *et al.*, “Towards Smarter Security Orchestration and Automatic Response for CPS and IoT,” in *Proc. 14th IEEE Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, Naples, Italy, 2023, pp. 298–302, doi: 10.1109/CloudCom59040.2023.00055.
- [16] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: rapid prototyping for software-defined networks,” in *Proc. ACM HotNets*, 2010, pp. 1–6, doi: 10.1145/1868447.1868466.